OpenRank Leaderboard: Motivating Open Source Collaborations Through Social Network Evaluation in Alibaba

Shengyu Zhao[1], Xiaoya Xia[2*]
Brian Fitzgerald[3], Xiaozhou Li[4], Valentina Lenarduzzi[4], Davide Taibi[4]
Rong Wang[5], Wei Wang[2†], Chunqi Tian[1]
[1]Tongji University, China — [2]East China Normal University, China
[3] Lero - Irish Software Research Centre, Ireland — [4]University of Oulu, Finland — [5]Alibaba Group, China
frank_zsy@tongji.edu.cn;xiaoya@stu.ecnu.edu.cn;bf@lero.ie;
xiaozhou.li@oulu.fi;valentina.lenarduzzi@oulu.fi;davide.taibi@oulu.fi;
tunan.wr@alibaba-inc.com;wwang@dase.ecnu.edu.cn;tianchunqi@tongji.edu.cn

ABSTRACT

Open source has revolutionized how software development is carried out, with a growing number of individuals and organizations contributing to open source projects. As the importance of open source continues to grow, companies also expect to grow thriving and sustainable open source communities with continued contributions and better collaborations. In this study, we applied the contribution leaderboard to seven open source projects initiated by Alibaba. We conducted a case study to investigate the perceptions and facts regarding how to motivate collaboration through gamification. Specifically, we employed a social network algorithm, OpenRank, to evaluate and steer developers' contributions. We validated the effectiveness of OpenRank by comparing it with other evaluation metrics and surveying developers. Through semi-structured interviews and project metric analysis, we found that the OpenRank Leaderboard can promote transparent communication environments, a better community atmosphere, and improved collaboration behavior.

KEYWORDS

Open source contribution, Social network, Leaderboard, Gamification

ACM Reference Format:
Shengyu Zhao[1], Xiaoya Xia[2] and Brian Fitzgerald[3], Xiaozhou Li[4], Valentina Lenarduzzi[4], Davide Taibi[4], Rong Wang[5], Wei Wang[2†], Chunqi Tian[1]. 2024. OpenRank Leaderboard: Motivating Open Source Collaborations Through Social Network Evaluation in Alibaba. In *46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24), April 14–20, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3639477.3639734

[*]Xiaoya Xia and Shengyu Zhao contributed equally to this research.
[†]Wei Wang is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE-SEIP '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0501-4/24/04…$15.00
https://doi.org/10.1145/3639477.3639734

1 INTRODUCTION

The increasing popularity of open source projects has led to the development of an ecosystem in which individuals and organizations can collaborate to create high-quality software that is available to all [1]. Unlike traditional software development activities that are conducted internally with designated tasks for specific individuals, open source development relies on voluntary work that requires extensive asynchronous communication and distributed collaboration [2]. With the rise of companies as significant users and even major players in the open source software ecosystem, they are increasingly taking the initiative to create, open source, and maintain their own projects as a way to contribute to the community and build technical influence in the industry [3]. Naturally, companies want their open source projects to flourish as thriving communities, receiving contributions from non-employees rather than just from their own employees. This requires attracting new developers to projects in the first place (magnetism) [4, 5] and subsequently retaining these developers over time (stickiness) [6–8]. To achieve this, factors such as continued contribution, open collaboration, and fostering a positive community atmosphere are essential for long-term sustainability [9].

While some previous work has studied the role of gamification in motivating open source contributions, this has involved students [10]. In this study, we present a real-world industrial practice implemented by Alibaba to encourage contributions to its open source projects. Our approach involves constructing a network algorithm that evaluates the contributions made by open source developers. Additionally, we leverage a monthly updated leaderboard to incentivize contributions and foster a sense of healthy competition among developers.

While previous work has explored methods for evaluating the efforts of open source contributors [11–13], to our knowledge none have approached the task from a social network perspective. The collaborative nature of open source development has long been recognized as essential to the success of OSS communities [1, 14], where communication, coordination, and social connections among developers are vital to producing high-quality software. With GitHub as the leading platform for social coding [15] and pull-based development [16], activities related to issues and pull requests (PRs) have become increasingly critical in open source software development. To leverage the power of social networks in open source development, we developed a network algorithm called

OpenRank and applied it to a collaborative network model, which serves as the core mechanism for our contribution leaderboard, allowing us to calculate individual contribution results and display them on the leaderboard, steering developers towards behaviors that foster good collaboration.

In October 2022, we launched OpenRank Leaderboard (ORL) in seven popular open source projects initiated and maintained by Alibaba. We employed quantitative and qualitative approaches to investigate the impact of social network evaluation-based leaderboards on open source collaborations.

Firstly, we aim to verify the soundness and effectiveness of our novel evaluation model by conducting surveys among contributors and making comparisons with other metrics:

RQ1: Can OpenRank Leaderboard (ORL) effectively reflect the contributions of open source developers?

Secondly, we intend to comprehend individual-level perceptions, and cognitive and behavioral changes by conducting semi-structured interviews with both project maintainers and contributors:

RQ2: How do individual developers react to OpenRank Leaderboard (ORL) and perceive its impact?

Finally, we seek to understand the influence that such a gamified leaderboard could bring to open source project collaborations through the analysis of project metrics data:

RQ3: How does OpenRank Leaderboard (ORL) impact the metrics of open source projects?

Our work contributes to both research and practice by:

- Introducing a new model for evaluating open source contributions that leverage social collaboration networks. Through developer feedback, we have demonstrated the effectiveness of this approach.
- Providing valuable insights into the impacts and perceptions of using a leaderboard as a gamification mechanism in company-led open source projects, both in terms of quantitative metrics and qualitative feedback.

The paper is structured as follows. Section 2 presents related work. Section 3 introduces the industry context. Section 4 presents the research method and results for each of the three research questions. Section 5 discusses the implications and threats, while Section 6 concludes the paper.

2 RELATED WORK

In this section, we will discuss the related work in software engineering and open source from three perspectives: gamification, collaboration networks, and contribution evaluation.

Gamification refers to the use of various game elements such as points, badges, levels, and leaderboards to enhance developer engagement and promote positive behaviors [17, 18]. Much of the research on gamification in software engineering has focused on domains such as software education [19, 20] or software usage [21, 22]. Since open source developers often contribute voluntarily without financial incentives, they may lose interest and leave eventually. Therefore, various forms of gamification have been discussed and applied in industry practices to motivate individual open source developers. The most commonly discussed approach is the bounty program [23, 24], which can raise high demands and time costs for backers. Although GitHub publishes project trending and ranking, it has not been applied to developer levels with specific operational goals.

Researchers have primarily focused on establishing collaboration networks in software development either on a file basis or a repository basis. Gonzalez-Barahona et al. [25] suggested employing nodes as modules in CVS repository while edges signify two modules that share common committers. Similarly, Yin et al. [26] developed a socio-technical network by utilizing connections in mailing lists and source files to forecast the sustainability of Apache incubator projects. Additionally, Surian et al. [27] constructed collaboration networks to recommend developers through a random walk algorithm. Our research contributes to the field by constructing a collaboration network based on Issues/PRs and applying it to actual industry scenarios to motivate collaborations in software projects.

Assessing contributions in software engineering involves measuring lines of code (LoC) or function points toward the final development of a software project [28]. The Walston-Felix model [29] defines productivity as the ratio of delivered source lines of code to total effort in man-months. However, with the emergence of open source projects, Gousios et al. [12] proposed a linear model that considers multiple action traces from development and communication activities to evaluate contributions. In previous research, we investigated identifying key contributors through mining activities in repositories [11]. However, we believe that the linear model is inadequate to reflect the social factors in OSS communities. Tsay et al. [30]'s study provides clear evidence that social factors are highly correlated with maintainers' contribution evaluation during PR review. Therefore, this study is a preliminary attempt to investigate the viability of using social networks to evaluate the contribution and value of developers.

3 INDUSTRY CONTEXT

3.1 Alibaba Open Source Contribution Leaderboard

Alibaba has adopted traditional DevRel (Developer Relations) operations to motivate developers from open source communities [31], including publishing technical blogs, organizing in-person events, and even running its Summer of Code Programs. However, these operating practices pose some challenges, including high operational costs and low conversion rates, as well as difficulties in persistence and RoI assessment. There is a need in the industry context for a quantifiable method to consistently motivate open source contributions. To address this, the company launched Alibaba Open Source Contribution Leaderboard[1] on October 1st, 2022, through which we adopted a network-based evaluation method, OpenRank, to evaluate contributions.

This research emerged from continued collaboration with the Alibaba Open Source Program Office (OSPO). The first and second authors held internships in the Alibaba OSPO while conducting this research, and participated in the launch of the contribution leaderboard. Seven projects created and continuously maintained by Alibaba were initially recruited[2] to participate in the program.

[1] https://opensource.alibaba.com/collection/contribution_leaderboard
[2] As of April 2023, a total of 20 projects have joined the leaderboard portal.

Table 1: Case Study Projects.

repo_name	participants	stars	age
alibaba/ice	207	17.5k	2016.11
alibaba/spring-cloud-alibaba	410	25.5k	2017.12
alibaba/DataX	409	13.4k	2018.01
alibaba/easyexcel	748	27.7k	2018.02
seata/seata	521	23.7k	2018.12
openyurtio/openyurt	111	1.5k	2020.05
fluid-cloudnative/fluid†	154	1.3k	2020.07

† Fluid community is co-initiated by Nanjing University and Alibaba.

These projects were all mature and popular projects that have been in existence for at least 2 years, with an average of 2 to 3 full-time committed Alibaba employees. Table 1 presents the project repository names, community participants on GitHub over the past year, the number of project stars, and the date that projects were created. The overall case study spans 12 months (April 2022 to April 2023).

Figure 1 depicts a snapshot of the leaderboard for the project *seata/seata*. The leaderboard is updated monthly and displays rankings of contributors, their GitHub IDs and avatars, and monthly contribution value scores calculated by OpenRank. Additionally, it highlights the rise and fall in contributions compared to the previous month and distinguishes new contributors and bots.

To increase visibility and understanding of the leaderboard, and reveal the calculation details among contributors, we employed several strategies. Firstly, we requested that project maintainers publish weekly reports in community channels to disseminate the results. Secondly, we displayed a badge linking to the leaderboard page for each respective project at the top of their repository README.md files. (An example is shown in Figure 2). Thirdly, we provided detailed network visualizations and calculation details to clarify the source of contributor value in terms of Issues/PRs (An example is shown in Figure 3). Finally, we included algorithm explanations on the leaderboard homepage to enhance transparency and understanding. However, as the effectiveness of the algorithm had not been validated before this study, we opted not to promote the leaderboard extensively through media or external channels. Despite this, the leaderboard has garnered a noteworthy level of attention among the case study participants, as evidenced by an average of 879 monthly page views since its launch.

3.2 Leaderboard Design Philosophy

In gamification, the actual design is more challenging than that of conventional software, and is a key factor that can ultimately affect developer behavior [32]. In this study, we utilized a social network model, OpenRank, as the core design for our contribution leaderboard, to explore the social aspects of open source contributions by constructing an effective measurement algorithm based on social collaboration networks. The core logic of OpenRank is inspired by PageRank, where more valuable web pages are more likely to be referenced by other web pages [33]. By calculating the centrality of developer nodes in collaboration networks, we sought to evaluate

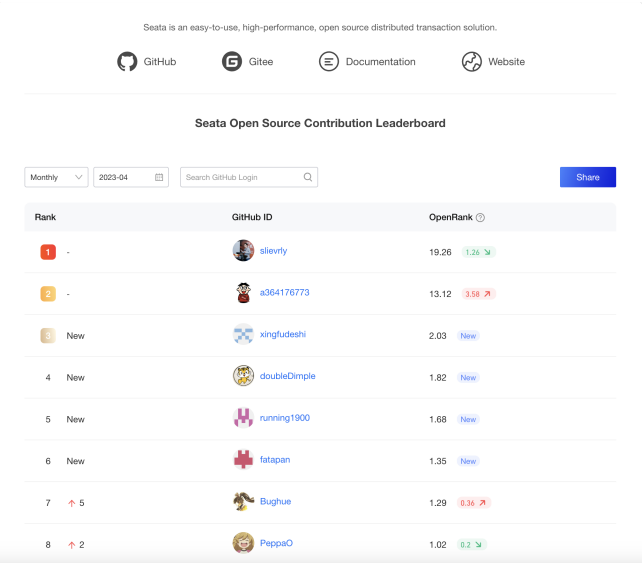


Figure 1: A snapshot of the leaderboard for the project *seata/seata* in April, 2023

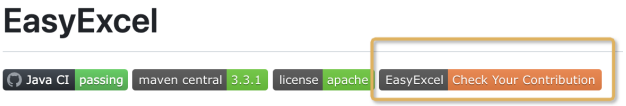


Figure 2: The badge displayed on top of the project *alibaba/easyexcel*'s README.md file

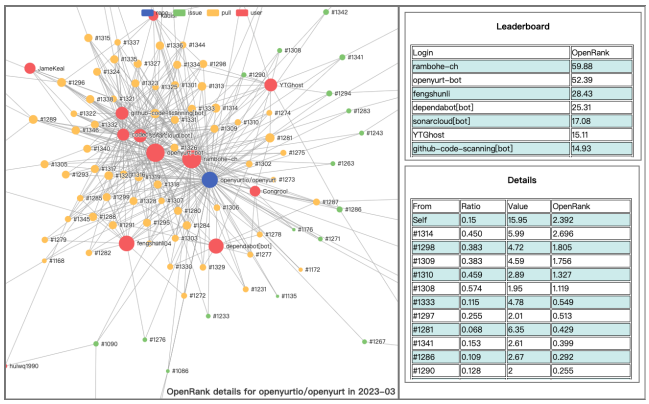


Figure 3: The detailed collaboration network for the project *openyurtio/openyurt* in March, 2023

the importance and commitment of a contributor toward an open source project.

While a leaderboard can easily cause the 'observer effect' [34], leading to negative behavioral changes among developers who become aware of the metric, a network algorithm with its complexity and non-linear characteristics makes it difficult to cheat by conducting bulk operations.

The design of this algorithm was aimed at creating a more engaging and rewarding experience for developers, promoting social collaboration and healthy competition, and ultimately leading to more sustainable and successful open source projects. We drew inspiration from the Apache Way [35], the governance philosophy of the long-standing Apache Foundation. We considered three principles in our design:

3.2.1 Open Communication. Under the Apache Way, open communication is a core tenet. According to this principle, communication that doesn't occur in public forums or mailing lists is considered not to have occurred.

In our algorithm, we apply this principle by considering GitHub as the primary platform and venue for open source community activities. Contributions that are not publicly documented on GitHub, such as daily chats in non-public instant messaging platforms, are not counted as valid contributions.

3.2.2 Earned Authority. The Apache Way emphasizes that every new developer who joins a community must earn their authority through their contributions to the project, rather than receiving special treatment based on their personal attributes or employment in other communities.

In our algorithm, we apply this principle by assigning an initial value of 1 to every newly-added developer to the collaboration network, regardless of their attributes. The centrality of all developers can only be increased through long-term deep involvement in the project. Additionally, the higher a developer's centrality, the greater their influence on related Issues/PRs. This reflects the principle that authority needs to be earned through contributions, and long-term contributions will increase a developer's decision-making influence.

3.2.3 Community Over Code. The Apache Way emphasizes that the community is more important than the code. A project with many developers who collaborate on long-term construction will always surpass a project with initial good quality but without continuous maintenance by the community.

In our algorithm, we prioritize social coding by focusing on collaborative relationships. We do not rely on precise calculation of the value of each contribution. Instead, we observe which Issues/PRs in which the core developers participate, which ones they like, and which other developers they like. The value judgment is no longer based on the specific content of the collaboration unit, but rather on the social relationships within the community.

3.3 Constructing OpenRank

The OpenRank algorithm[3] is a variant of the PageRank algorithm, but customized for open source project collaboration units such as Issues and Pull Requests, with developers as nodes.

3.3.1 Network model. The basic collaborative network model is presented in Figure 4. It depicts a scenario in which developers collaborate on Issue and PR nodes within a repository on GitHub, with different types of collaborative behaviors taking place. Notably, the initial value of Issue and PR nodes will be influenced by developers adding ***thumbs-up*** emojis to the body of Issues and PRs.

[3]The implementation of OpenRank using Pregel framework as a plugin of Neo4j graph database is open source at: https://github.com/X-lab2017/openrank-neo4j-gds

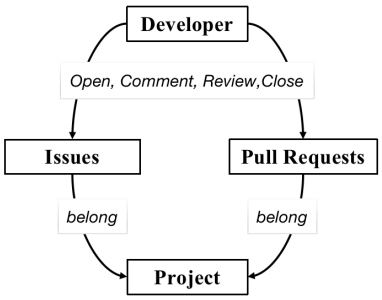


Figure 4: The basic network model

3.3.2 Algorithm. We utilized an algorithm that is similar to PageRank to determine the centrality of both Issues/PRs and developers. This measure of centrality was used as an indicator of the contribution value. Unlike PageRank, the centrality of each node in the OpenRank algorithm is not solely based on the structure of the collaboration network, but also takes into account the intrinsic value of the node. This unique feature means that the OpenRank algorithm is not a Markov process, and therefore the convergence of OpenRank necessitates additional proof, which is included in the supplementary materials.

In general, the OpenRank value of each node v_i in each iteration is represented by the following equation:

$$v_i = (1 - a_i) \sum_{j=1}^{|V|} \frac{w_{ji}}{d_{oj}} v_j + a_i v_0$$

Where v_0 represents the initial value of the node, a_i represents how much the node depends on its initial value, d_{oj} is the weighted out-degree of node j, w_{ji} is the weight of the edge from node j to node i. If we organize the normalized weights w_{ji}/d_{oj} into matrix S and a_i into matrix A as diagonal values, according to the convergence proof, the OpenRank values of all nodes will converge to a vector as follows:

$$\begin{aligned} \boldsymbol{v} &= \lim_{k\to\infty} \boldsymbol{v}^{(k)} \\ &= \lim_{k\to\infty} [\boldsymbol{ASv}^{(k-1)} + (\boldsymbol{E} - \boldsymbol{A})\boldsymbol{v}^{(0)}] \\ &= \lim_{k\to\infty} [(\boldsymbol{AS})^k \boldsymbol{v}^{(0)} + \sum_{t=0}^{k-1} (\boldsymbol{AS})^t (\boldsymbol{E} - \boldsymbol{A})\boldsymbol{v}^{(0)}] \\ &= (\boldsymbol{E} - \boldsymbol{AS})^{-1} (\boldsymbol{E} - \boldsymbol{A})\boldsymbol{v}^{(0)} \end{aligned}$$

3.3.3 The Initial Value of Nodes. The OpenRank algorithm takes into account the initial value of each type of node during centrality calculations. Since OpenRank performs calculations based on monthly data, the initial value of a node is inherited from the previous month if the node participated in the previous month's calculation and already obtained a centrality value. However, if a node does not have a centrality, its initial value will be set to 1. Specifically,

Table 2: AHP Evaluation Matrix and Results.

Behavior	Issue/PR Comment	PR Review	Close Issue	Close PR	Open Issue	Open PR	Eigenvector	Weight(%)
Issue Comment	1	0.5	0.5	0.333	0.25	0.2	0.401	5.252
PR Review	2	1	0.5	0.5	0.333	0.2	0.567	7.427
Close Issue	2	2	1	0.5	0.333	0.25	0.742	9.712
Close PR	3	2	2	1	0.5	0.333	1.222	14.695
Open Issue	4	3	3	2	1	0.333	1.698	22.235
Open PR	5	5	4	3	3	1	3.107	40.679

Table 3: AHP Consistency Check.

λ_{max}	CI	RI	CR	Result
6.169	0.034	1.25	0.027	Pass

*Consistency check passes if $CR < 0.1$

- For Issue/PR nodes, the initial value is increased based on developers adding *thumbs-up* reactions. The value is multiplied by the *developer-who-adds-thumbs-up*'s contribution proportion from the previous month. That is to say, a developer with more contributions to the project has a greater influence on the value of issues and PRs.
- If a PR node is merged in the current month, the initial value of the PR node is additionally increased by 50%.

3.3.4 Proportional Dependency on Initial Value. When determining the centrality of different types of nodes within a network, it is important to consider both their initial value (i.e., their value independent of their connections within the network) and the value they receive from their interactions with other nodes (i.e., the network value transfer).

For all nodes in the network, 15% of their centrality is based on their initial value, while 85% is determined by their interactions and connections within the network. This parameter set refers to the default damping factor [36] of PageRank.

3.3.5 Proportional Network Value Transfer. After setting the initial value of each node, the algorithm must consider value transfer to ensure convergence and the value transfer proportion for each type of node should add up to 1. Developers and Issues/PRs nodes are connected by the edges of different collaboration behaviors including open, comment, review, and close. These behaviors should be weighted differently since they represent different efforts from developers. To determine the weights, we applied the Analytic Hierarchy Process (AHP), which is commonly used in operations research [37]. Weight determination is subjective to experts' experience, especially tenured developers in open source communities. Thus, we presented the AHP evaluation matrix (as shown in Table 2) to the seven maintainers of participated projects, instructed them on the 5-point scale AHP evaluation, and then calculated the final value by the geometric mean. Using AHP analysis, we obtained the weights for each collaboration behavior, as presented in the last column of Table 2. It can be observed in Table 3 that this set of parameter settings passed the consistency test.

4 RESULTS

4.1 RQ1: Can ORL effectively reflect the contribution of open source developers?

To address this RQ, we rely on objective comparisons with other metrics and subjective assessments from developers. Firstly, we compared the results calculated by OpenRank with other single-dimensional metrics. Then, we administered a questionnaire to a sample of 527 open source developers who had participated in the seven projects within the last year.

4.1.1 Comparison. We use the Spearman correlation to compare the ranking results of OpenRank between Commit, LoC, Open Issue, Open PR, PR Review, and Issue/PR Comment, respectively. To mitigate the long-tail effect, we selected the top 200 developers ranked by OpenRank of all the projects. The results show that OpenRank exhibits significant correlations with all the metrics ($p < 0.01$). Except for a relatively weak positive relationship with Open Issue ($r = 0.37$), OpenRank exhibits a moderate positive relationship with all other metrics ($0.63 \leq r \leq 0.66$).

Then, we further explored the cases that show significantly different results under various metrics. Table 4 presents two developers whose ranks differ in OpenRank compared to those in Commit or LoC. In *spring-cloud-alibaba (sca)*, *DanielLiu1123* ranks 5th measured by OpenRank, with only 5 commits contributed. The reason is that *DanielLiu1123* is a community maintainer for problem-solving. The 62 comments are distributed among 50 different Issues/PRs, making this developer a crucial coordinating node in the collaborative network, and significantly boosting the OpenRank value. On the other hand, a *seata* contributor *goodBoyCoder*, ranks 51st in OpenRank despite contributing 47 commits and over 7,000 lines of code. This is because he did not actively participate in community discussions. According to the maintainer, this developer participated in the project through the Summer of Code Program, completing the entire program's work with 3 PR submissions but rarely engaging in collaboration and discussions on other community Issues and PRs. Additionally, this contributor did not continue contributing to the project after the summer program, resulting in attrition.

4.1.2 Survey. We injected the developer's GitHub ID and project information into each questionnaire, developers who were active in multiple projects were given multiple questionnaires. We initially inquired about the contributor's community role: maintainer, committer, reviewer, or without any role. We then inquired about whether the project's leaderboard results over the past 12 months

Table 4: Comparison of OpenRank with other developer contribution evaluation metrics: Selected Cases

Developer	Project	$OpenRank_{rank}$	$Commit_{rank}$	LoC_{rank}	Comment	Open Issue	Open PR	PR Review
DanielLiu1123	*sca*	9.83_5	5_{20}	381_{15}	62	3	2	11
GoodBoyCoder	*seata*	0.95_{51}	47_{15}	7275_2	0	0	3	1

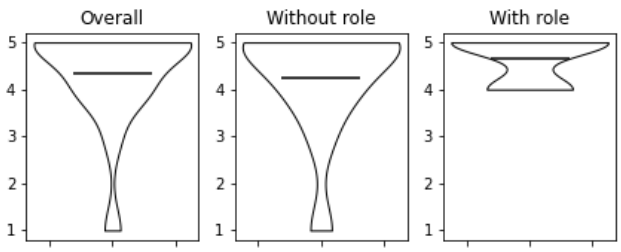


Figure 5: Distribution of Survey Score

Table 5: The Average Score of the Survey.

Type	Dev. Count	Avg. Score
Overall	63	4.35
Without role	46	4.24
With role	17	4.65
OpenRank weighted	63	4.88

aligned with their expectations, using a Likert scale question [38] that ranges from 1 (completely unmet) to 5 (completely met). Additionally, we provided an optional open-ended question for respondents to expand upon any instances where the results did not match their expectations, or to provide recommendations for improving the calculation method. Lastly, we inquired as to whether they would be interested in participating in a follow-up interview.

The sample consisted of 527 individuals, from whom we obtained 63 responses, rendering a 12% response rate, which is consistent with response rates reported in comparable software engineering research surveys [39, 40].

17 responses were from developers who held community roles, and 46 responses were from developers without community roles. Figure 5 illustrates the distribution of the scores concerning the Likert scale question, and the results showed that 55 questionnaires received a grade of 4 or higher, representing more than 87% of the total responses. All developers with community roles gave scores of 4 or higher, with 65% providing a score of 5.

To effectively capture cognitive differences among developers with varying contribution levels, we use the developer cumulative OpenRank value over the past year for each project to calculate a weighted average. The average scores are presented in Table 5. As observed, the overall average score is 4.35, while the average score for developers without any community role is 4.24, the average score for developers with community roles is 4.65, and the OpenRank-weighted average result is 4.88. Therefore, we can conclude that the overall results align with developers' expectations, and we also infer that developers who possess a deeper understanding of the community (with community roles or higher contributions) are more agreeable with the results.

Three developers who gave a score of 1 provided feedback about issues related to the project features, which suggests that the low scores received from them may not be directly linked to the contribution measurement process, but rather the project's overall functionality, or their personal experience with it.

Notably, one of the respondents who held the maintainer role provided feedback regarding a developer with whom he was not familiar but who appeared prominently on the leaderboard. Upon closer inspection of the repository data, it was discovered that this developer had consistently acted as an issue replier, resolving community issues and promptly closing the related issues. The developer's contribution had been overlooked because maintainers did not pay much attention to resolved issues. Meanwhile, the collaborative relationships formed through communication with various developers while resolving multiple issues enhanced this developer's OpenRank values, thereby allowing otherwise unnoticed contributions to be recognized through ORL.

We have also received feedback suggesting that, as a measure of contribution, the content of the contribution should be considered, rather than just the collaboration behavior and relationships. We also addressed this issue during the interviews with developers in RQ2, so we will elaborate on it further in Section 4.2.

> OpenRank shows a significant correlation with other metrics and highlights the work of coordination among different Issues and PRs. Overall, developers endorse the results of using OpenRank to measure contributions, and those with community roles and higher contributions have a higher level of endorsement for the results.

4.2 RQ2: How do developers react and perceive ORL?

To address this RQ, we interviewed maintainers and contributors to gain insight into any behavioral changes and to gauge overall sentiment towards ORL. The interviews begin with an initial introduction by one researcher, and focus on the following key open-ended questions:

- What are your feelings toward ORL?
- Have you attempted to improve your contribution value or rank? If so, how?
- How do you believe ORL influences your behavior?
- What suggestions or vision do you have for ORL?

We first conducted interviews with all 7 project maintainers (M1-M7). Then, based on the previous survey, we recruited an additional 11 external contributors (C1-C11), resulting in a total of 18 interview participants. Each interview lasted between 30-45 minutes and was conducted and transcribed in Chinese, as all interviewees were Chinese. To analyze the interview data, we utilized thematic coding [41]. The first and second authors independently coded the themes, met to reconcile any disagreements, and developed the final results. As a final step, we conducted member-checking which is recommended for helping ensure the validity of qualitative research [42], and presented the results to the 18 interviewees for confirmation to ensure that our coding was clear and did not create any ambiguity or misunderstanding.

Among the 11 contributors we interviewed, 3 were student contributors, 7 were full-time employees, and 1 was a freelancer. There were 8 male and 3 female contributors. All 7 project maintainers (who were Alibaba employees) were male. 7 contributors recruited held Top 10 positions on the leaderboard and 4 contributors ranked lower (outside Top 10 each month).

We present the results from two perspectives: developer behavioral changes influenced by ORL and developer cognitive perception toward ORL.

4.2.1 Behavioral Influence. We found evidence of specific behavioral changes in the interviewees. Firstly, **developers are more willing to document their discussions on GitHub Issues (13 mentions)**, rather than keeping them private in Instant Messaging platforms. As C2 stated: *"Valuable information can easily be lost in casual and personal conversations, whereas GitHub provides a public forum to document this information and retrieve it in the future. ORL motivates me to cultivate this habit and be more proactive on GitHub."*

From RQ3 we have observed a significant increase in the number of *thumbs-up* reactions. Developers also mentioned their psychological changes during the process of **actively adding and receiving *thumbs-ups* through the interactions on GitHub (9)**: *"I was not accustomed to doing this, but ORL mentioned that thumbs-ups would affect the value of PR. As a result, I consciously add thumb-ups to content that I think is good. My motivation comes from the fact that this behavior will have a substantial impact on the PR author.* (C2); On the other hand, C7 noted that *"receiving thumbs-ups can bring a sense of achievement, and emoji interaction is another form of 'read', signifying that I have seen this message and think it's good."*

Contributors in the community who have not obtained tenure tend to choose to **collaborate with influential developers with high OpenRank to increase their value (3)**, for example, proactively reviewing their code, or actively participating in discussion on issues they are involved in. In contrast, it is interesting to note that M1, whose rank is high, stated that he sometimes **consciously chooses to condense his behaviors to narrow the gap (1)**. *"For example, if I make a comment and then want to add something later, in the past, I might have added a new comment. Now I click the edit button - GitHub supports modifying the comment - because I don't want to look like I'm still pursuing that score - my score is already high enough."*

Maintainers are more motivated to **propose task-oriented issues and assign them to suitable developers (2)** to solve due to the presence of the contribution leaderboard. As M1 described it: *"Assigning issues used to be a burden in open source development due to voluntary contributions. However, active developers on the leaderboard now have a strong desire to increase their contributions, making them more willing to accept and work on assigned project tasks."*

> To summarize, we found in the overall **community interactions**, the behavioral influence is:
> - Shifting discussions from private conversations to transparent GitHub issues.
> - Using emojis to express appreciation and confirm that messages have been read.
> - Developers without tenure tend to collaborate with those who have a high OpenRank.
> - Tenured developers may choose to condense their efforts to narrow the gap.
> - Maintainers become confident to raise tasks and assign them to specific contributors.

When submitting pull requests, developers mentioned that they would **split large pull requests into smaller ones (8)**, and frequently stated that the main benefit was facilitating code reviews, whereas increasing the contribution value was a side bonus. As a developer with commit permission, M7 mentioned that the biggest change he made was to **avoid directly committing to the code repository (5)**. Even small changes are submitted in the form of pull requests and **waiting for others to review and merge (3)**. This is because commits do not count towards any contribution to the algorithm, and directly merging one's pull request does not significantly increase the score compared to having others review and merge it.

M2 mentioned that there appeared to be contributors who **submit small patches such as typo fixes and dependency upgrades (3 cases found)**. We asked the maintainer if he was bothered by these small patches. His response was *"Not at all. These small contributions are often overlooked because they are so trivial, however, they are helpful to the project."*

> To summarize, regarding **project code development**, the behavioral influences are:
> - Splitting large code change patches into small PRs.
> - Avoiding directly committing to the code repository.
> - Waiting for others to review and merge the PR.
> - Addressing small fixes such as typo fix and dependency upgradation.

4.2.2 Perceptions toward ORL. Most developers agreed with using graph network algorithms to evaluate open source contributions (11 out of 18, the remaining did not mention this point). As M5 commented, *"Overall, the rankings are in line with my perception of the community members. Those who have been active with long-term contributions are on the list."* He also envisaged **combining the results with community incentive operations (3),** *"We often award medals or send gifts to outstanding contributors. In the past,*

these were based on maintainer personal judgment, but OpenRank results can serve as a good quantitative reference."

When discussing their understanding of the OpenRank algorithm, some developers believe that **the complexity of the network algorithm has resulted in a higher cost of understanding how to quickly increase their score (4)**. *"If there was only a single dimension to evaluate contributions, it might tempt people to do things that match the evaluation index, just like a Key Performance Indicator (KPI)."* (C3)

When asked for suggestions, two interviewees suggested that the value of the content in issues and PRs should be taken into account. However, one of them also stated that for long-term contributors, both valuable PRs and small fixes would be counted, and they should eventually reach a balanced state. New contributors, on the other hand, tend to provide only small contributions. Therefore, the evaluation of contributions from newcomers may be biased, **but also encouraged by maintainers (1)**.

We found that about half of the interviewees (8 interviewees, of whom 2 were maintainers) stressed that they did not change their behavior solely to increase their score based on the algorithm's parameter settings. Instead, they based their actions on *"what the project needs in terms of contributions and what I can do."*(C5) **The presence of the leaderboard is more of a self-assessment channel for their contributions (8).** For example, avoiding directly committing changes to the code repository, and waiting for others to review the code are standard practices for distributed collaboration [16, 43]. However, contributors with committed permission often ignore these rules for convenience and quickness. Not developing such a habit when submitting small patches may finally lead to negligence in significant changes in the future. ORL acts more as a mechanism to alert behaviors at certain moments.

In conclusion, developer perceptions toward ORL are that:

- The results of contribution leaderboards can be combined with community awards and rewards.
- The complexity of network algorithms increases the cost of foul play in gamification.
- Not evaluating the value of PR content is a form of encouraged bias towards newcomers.
- The presence of the leaderboard is more of a self-assessment channel while making contributions.

4.3 RQ3: How does ORL impact open source collaborations?

To determine the extent of the impact of the launch of ORL on the projects, we conducted a comparison of various metrics over six months before and after the launch. This yielded a total of 12 months of data for each project and metric. The initial six months of data (2022/04/01-2022/09/30) represent the results obtained before ORL launch, while the subsequent six months of data (2022/10/01-2023/04/01) represent the results obtained after the launch.

We observe project metrics from four dimensions: Issues, Change Requests, Reactions, and Community Size. Among them, the metrics related to Issues and Change Requests come from the CHAOSS [44, 45] Evolution working group. CHAOSS is an open source initiative that focuses on creating metrics to better understand open source community health. For Community Size, we observe the changes in the number of new contributors to the project (i.e. the number of developers whose new PRs have been merged) and the changes in the total number of participants (i.e. the number of developers who have participated in project collaboration). For Reactions, we observe the changes in the number of *thumbs-up* emojis in the project. All the metrics are implemented and analyzed using OpenDigger[4], an open source project that provides data analyzing and modeling toward the open source software ecosystem.

We utilized Regression Discontinuity Design (RDD) to analyze the aforementioned metrics. RDD is a quasi-experimental method widely adopted in gamification mechanisms to evaluate the effectiveness of mechanism design [46]. By observing the changes in the level and slope of the fitted equation of metrics before and after the mechanism intervention, RDD can determine whether the experiment had a significant impact on the metrics. This method has been applied in several software engineering-related studies to track metric changes over some time [47–49].

We conducted an RDD analysis on the above metrics and defined the following regression model to observe changes in the level and slope of linear regression equations before and after the gamification mechanism:

$$Y = \alpha + \beta \cdot T + \gamma \cdot G + \delta \cdot A + \eta \cdot C + \varepsilon_i$$

In this context, T represents **time** in months since the start of the observation, G is a binary flag indicating the **gamification** mechanism (0 before the launch, 1 after the launch), A represents the number of months **after** gamification (0 before the launch, incrementally increasing after the launch). while C is a control variable. Notably, the project development exhibits strong seasonality, such as heightened activity during summer vacation when students participate in open source communities, and decreased activity during the Chinese New Year. Given these factors correlate significantly with monthly changes in the number of collaborative events, we used the periodical collaborative events across all Chinese company-led open source projects during the observation period as a **control** variable to eliminate the impact of seasonal changes on the observed project metrics. The evolution curve of the control variable is shown in Figure 6 (a). The Chinese company-led open source projects are also collected and labeled in OpenDigger.

The regression analysis comprises two linear regressions - one before and one after the launch of the gamification mechanism. The parameter γ denotes the change in the level after the launch, while δ represents the change in slope after the launch. Therefore, in the analysis of this issue, we are particularly interested in the results of the two values, γ and δ.

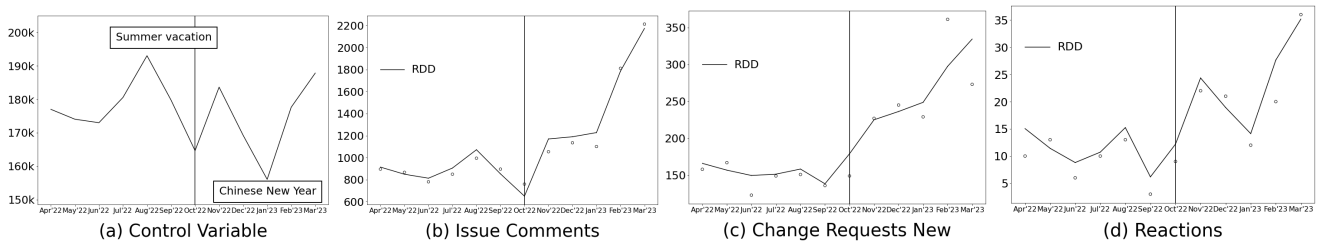
To observe the impact of ORL on various project metrics, we conducted separate RDD analyses for each metric, Table 6 presents the detailed results including the model fitness (R^2) in the RDD analysis, and the changes in level (γ) and slope (δ) brought about by the gamification mechanism. Additionally, the table displays the average improvement in each metric over the six months before and after the launch. In Figure 6, we present several evolution curves

[4]https://github.com/X-lab2017/open-digger

Table 6: RDD Analysis Result of Project Metrics

Category	Project Metric	R^2	RDD Analysis Level(γ)	RDD Analysis Slope(δ)	%Average Improvement After Gamification
Issues	Issues New	0.83	28.13 †	9.27 †	18.05%
	Issue Comments	0.98	44.43 †	255.66 ***	52.75%
Change Requests ‡	Change Requests New	0.83	62.54 †	32.32 **	67.87%
	Change Requests Accepted	0.83	43.43 †	25.80 †	88.73%
	Change Requests Reviews	0.87	7.50 †	19.66 †	76.95%
Reactions	Reactions	0.90	16.05 **	4.22 ***	118.18%
Community Size	New Contributors	0.66	44.44 **	3.51 †	27.50%
	Participants	0.92	78.48 **	3.41 †	-1.77%

†$p > 0.05$, **$p < 0.05$, ***$p < 0.01$
‡ According to CHAOSS, "Change Requests" corresponds to "Pull Requests" in the case of GitHub.



(a) Control Variable (b) Issue Comments (c) Change Requests New (d) Reactions

Figure 6: Evolution of project metrics from April 2022 to March 2023 on the (a) Control Variable, (b) Issue Comments, (c) Change Requests New, (d) Reactions

on metrics that demonstrate apparent increases after the launch, which are: Issue Comments, Change Requests New, and number of Reactions.

In terms of metrics related to Issues, there has been a notable surge in the number of comments. The comments count remained relatively consistent during the first six months but underwent a significant increase following the launch of ORL. This growth rate was remarkable, increasing by a staggering 255 times. Even though the initial volume of comments was already considerable, there was still an average growth of 52.75%. The results are highly significant, as evidenced by the p-value, indicating a clear impact of ORL.

In terms of metrics related to Pull Requests, it was found that the number of newly created PRs had a notable correlation, exhibiting a growth rate of 32.32% and an ultimate average increase of 67.87%. This discovery aligns with the interview results obtained from RQ2, where a certain number of developers submit a lot of small fixes through PRs to improve their OpenRank standings.

In terms of reactions, the total count of *thumbs-up* reactions is noteworthy in both level and slope, resulting in an average growth rate of 118.18%. Since developers did not typically add reactions to each other's issues or PRs before the launch of ORL, this metric is undoubtedly the most prominent one affected. In terms of the design of the mechanism, reactions not only possess general social properties but also assist the algorithm in adjusting the weights of issues and PRs. Therefore, changes in this metric not only reveal developers' acceptance of the algorithm but also demonstrate that developers were actively adjusting the weight of issues and PRs that they believe are valuable, which also reflects our evaluation philosophy of earned authority.

However, in the two metrics related to community size, there was no significant correlation evident in the slope, and the final average growth data was relatively modest. This finding is consistent with the lack of extensive media publicity for ORL, which did not attract a noticeable increase in new external developers to participate in the projects. Therefore, the improvement in other collaboration-related statistical metrics more accurately illustrates the effectiveness of the mechanism, as more collaborative events occurred within a relatively stable community size.

> Given the lack of media publicity, ORL has been inadequate in attracting new external developers. However, it has had a noticeable impact on developers who are already involved in the projects. Specifically, it has significantly promoted daily activities such as praising each other's work, participating in issue discussions, and submitting change requests through PRs.

5 DISCUSSION

5.1 OpenRank and OSS Contribution Evaluation.

Based on our research findings, the OpenRank algorithm, which relies on collaborative networks, can accurately quantify developer contributions in open source projects. Developers who are highly

involved and engaged in the project are more likely to be satisfied with the algorithm's results. Some other points discussed around the algorithm include:

(1) Taking advantage of the unique characteristics of graph network algorithms to investigate collaborative relationships, previously overlooked contributors can now be brought to the fore. An excellent illustration of this is the obscure issue replier mentioned in RQ1.

(2) Some respondents expressed concern about potential score manipulation for personal gain, but the algorithm complexity makes this difficult. Moreover, malicious behavior in open collaboration can permanently tarnish one's reputation, which is a deterrent for developers to manipulate scores solely for ranking purposes.

(3) Some respondents suggested a more refined calculation of Issues and Pull Requests. This is a challenging and complex task that may require consideration in the future. However, some maintainers argued that the less refined calculation better meets the demands of their community operation, as it is more conducive to newcomers and aligns with the community's objective of attracting new developers which has been widely recognized as a critical issue [4, 5].

Overall, we believe that the OpenRank algorithm is a valuable tool that effectively measures developers' contributions to open source projects and has not produced any significant negative outcomes.

5.2 Leaderboard in Open Source Collaboration.

Based on our research, gamifying the contribution leaderboard through OpenRank can positively impact open source developers' collaboration habits (RQ2). These changes include:

(1) Developers are more likely to split larger, interdependent PRs into smaller, independent PR units. This approach makes PRs more reviewer-friendly, resulting in shorter review and resolution periods.

(2) Developers with repository writing permissions are now avoiding committing small changes directly to the repository or their PRs. Code merging without community review is risky and leads to a lower score on the leaderboard.

(3) The community is receiving more small fixes from external contributors, such as typos or dependency version upgrades. Although these fixes may seem insignificant, maintainers welcome them as they contribute to the continuous refinement of the project. With the existence of the leaderboard, developers who were once just users are now submitting PRs to fix these minor issues.

(4) The use of *thumbs-up* emojis to adjust the value of issues and PRs not only aligns the algorithm's results with developers' value judgments but also encourages other developers to add *thumbs-ups* to high-quality collaborative content. Developers have reported that the community's overall atmosphere has become friendlier, and social connections between developers have strengthened.

Overall, our study supports the potential for gamifying the contribution leaderboard through OpenRank to positively influence open source developer behavior, leading to a more cooperative and supportive community culture.

5.3 Threats to Validity

Firstly, an important risk is that the project's behavioral data will fluctuate according to its development plan and lifecycle, and the changes observed in the project indicator dimensions cannot be entirely attributed to the existence of the leaderboard. To control these variables as much as possible, we selected mature and stable projects and added the overall data fluctuations of Chinese projects in a year as a parameter in the RDD analysis. We acknowledge that selecting a longer lifecycle comparison, such as using the natural year as the unit, may be more effective, but the selected period in this study has already achieved good experimental results.

The evaluation of developers' contributions in the OpenRank algorithm is based on collaborative events in Issues and PRs and does not consider the value of content in Issues and PRs, which is different from traditional models that are based on workload. We received feedback from developers suggesting that the value of PRs themselves should be taken into account. However, as discussed in RQ2, another perspective is that for long-term contributors, the ultimate value of Issues/PRs will return to the mean level. In this work, we differentiate the value of Issues/PRs through *thumbs-up* reactions, but a more precise and scientific evaluation of the value of PRs itself is a topic worthy of future exploration. Additionally, our mechanism does not consider contributions outside of the GitHub platform, which is also one of the value orientations of open collaboration in our case study - what does not happen on GitHub is as if it did not happen.

Finally, during the interviews, we also found that some respondents did not have a strong perception of the calculation details due to the complexity of the graph network algorithm. However, calculations based on single dimensions may lead to distorted action. We provided a value-oriented explanation of the algorithm, and the leaderboard did indeed bring some positive effects, such as more detailed code reviews, more reactions, and more transparent discussions. We believe that if the leaderboard is combined with rewards, it will generate more interesting impacts and fun facts, including possible negative effects that we hope to showcase in future work.

6 CONCLUSION

In summary, our study presents Alibaba's industrial practice of encouraging contributions to its open source projects through the use of a network algorithm called OpenRank and a monthly updated contribution leaderboard. We investigated the impact of gamified leaderboards on open source project collaboration through surveys, analysis of project metrics data, and semi-structured interviews with project maintainers and contributors. The results show that the gamification of a contribution leaderboard can impact open source developers towards better collaboration habits and a more harmonious community atmosphere with stronger social connections. The study contributes to research and practice by introducing a new method for evaluating open source contributions that leverage social collaboration networks and providing valuable insights into the impacts and perceptions of using leaderboards as a gamification mechanism in company-led open source projects. In terms of future work, we plan to refine the OpenRank algorithm, for instance, incorporating different factors in the value calculation

of issues and pull requests. This study focused on seven projects initiated and maintained by Alibaba, so in the future, it would be useful to explore whether the same results are consistent across a wider variety of projects. Finally, it would also be important to investigate whether the positive effects of gamified leaderboards are sustainable over a longer period.

7 ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China Grants 62137001, 62277017, and 61977026. We express our gratitude to all the participants who dedicated their time to respond to our survey and interviews, special thanks to Zhenhu Li for his assistance in conducting interviews. The supplementary material of this work can be found at: https://github.com/X-lab2017/ICSE-OpenRank-material.

REFERENCES

[1] Steven Weber. The success of open source. In *The Success of Open Source*. Harvard University Press, 2005.
[2] Joseph Feller and Brian Fitzgerald. *Understanding open source software development*. Addison-Wesley Longman Publishing Co., Inc., 2002.
[3] Junxiao Han, Shuiguang Deng, David Lo, Chen Zhi, Jianwei Yin, and Xin Xia. An empirical study of the landscape of open source projects in baidu, alibaba, and tencent. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 298–307. IEEE, 2021.
[4] Huilian Sophie Qiu, Yucen Lily Li, Susmita Padala, Anita Sarma, and Bogdan Vasilescu. The signals that potential contributors look for when choosing open-source projects. *Proceedings of the ACM on Human-Computer Interaction*, 3(CSCW):1–29, 2019.
[5] Igor Steinmacher, Marco Aurelio Graciotto Silva, Marco Aurelio Gerosa, and David F Redmiles. A systematic literature review on the barriers faced by newcomers to open source software projects. *Information and Software Technology*, 59:67–85, 2015.
[6] Andreas Schilling, Sven Laumer, and Tim Weitzel. Who will remain? an evaluation of actual person-job and person-team fit to predict developer retention in floss projects. In *2012 45th Hawaii international conference on system sciences*, pages 3446–3455. IEEE, 2012.
[7] Fabio Calefato, Marco Aurelio Gerosa, Giuseppe Iaffaldano, Filippo Lanubile, and Igor Steinmacher. Will you come back to contribute? investigating the inactivity of oss core developers in github. *Empirical Software Engineering*, 27(3):76, 2022.
[8] Bin Lin, Gregorio Robles, and Alexander Serebrenik. Developer turnover in global, industrial open source projects: Insights from applying survival analysis. In *2017 IEEE 12th International Conference on Global Software Engineering (ICGSE)*, pages 66–75. IEEE, 2017.
[9] Johan Linåker, Efi Papatheocharous, and Thomas Olsson. How to characterize the health of an open source software project? a snowball literature review of an emerging practice. In *Proceedings of the 18th International Symposium on Open Collaboration*, pages 1–12, 2022.
[10] Guilherme C Diniz, Marco A Graciotto Silva, Marco A Gerosa, and Igor Steinmacher. Using gamification to orient and motivate students to contribute to oss projects. In *2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, pages 36–42. IEEE, 2017.
[11] Xiaoya Xia, Zhenjie Weng, Wei Wang, and Shengyu Zhao. Exploring activity and contributors on github: Who, what, when, and where. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, pages 11–20, 2022.
[12] Georgios Gousios, Eirini Kalliamvakou, and Diomidis Spinellis. Measuring developer contribution from software repository data. In *Proceedings of the 2008 international working conference on Mining software repositories*, pages 129–132, 2008.
[13] Gregorio Robles, Jesús M González-Barahona, Carlos Cervigón, Andrea Capiluppi, and Daniel Izquierdo-Cortázar. Estimating development effort in free/open source software projects by mining software repositories: a case study of openstack. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, pages 222–231, 2014.
[14] Vishal Midha and Prashant Palvia. Factors affecting the success of open source software. *Journal of Systems and Software*, 85(4):895–905, 2012.
[15] Laura Dabbish, Colleen Stuart, Jason Tsay, and Jim Herbsleb. Social coding in github: transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 conference on computer supported cooperative work*, pages 1277–1286, 2012.
[16] Georgios Gousios, Martin Pinzger, and Arie van Deursen. An exploratory study of the pull-based software development model. In *Proceedings of the 36th international conference on software engineering*, pages 345–355, 2014.
[17] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work?–a literature review of empirical studies on gamification. In *2014 47th Hawaii international conference on system sciences*, pages 3025–3034. Ieee, 2014.
[18] Oscar Pedreira, Félix García, Nieves Brisaboa, and Mario Piattini. Gamification in software engineering–a systematic mapping. *Information and software technology*, 57:157–168, 2015.
[19] Panagiotis Fotaris, Theodoros Mastoras, Richard Leinfellner, and Yasmine Rosunally. Climbing up the leaderboard: An empirical study of applying gamification techniques to a computer programming class. *Electronic Journal of e-learning*, 14(2):94–110, 2016.
[20] Chung-Ho Su. The effects of students' motivation, cognitive load and learning anxiety in gamification software engineering education: a structural equation modeling study. *Multimedia Tools and Applications*, 75:10013–10036, 2016.
[21] Ali Darejeh and Siti Salwah Salim. Gamification solutions to enhance software user engagement—a systematic review. *International Journal of Human-Computer Interaction*, 32(8):613–642, 2016.
[22] Carlos J Costa and Manuela Aparicio. Gamification: Software usage ecology. *Gamification: software usage ecology*, (1):92–100, 2018.
[23] Tetsuya Kanda, Mingyu Guo, Hideaki Hata, and Kenichi Matsumoto. Towards understanding an open-source bounty: Analysis of bountysource. In *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 577–578. IEEE, 2017.
[24] Jiayuan Zhou, Shaowei Wang, Haoxiang Zhang, Tse-Hsun Chen, and Ahmed E Hassan. Studying backers and hunters in bounty issue addressing process of open source projects. *Empirical Software Engineering*, 26(4):81, 2021.
[25] Jesús M González-Barahona, Luiz Lopez, and Gregorio Robles. Community structure of modules in the apache project. In *Proceedings of the 4h International Workshop on Open Source Software Engineering*, pages 44–48. IET, 2004.
[26] Likang Yin, Zhuangzhi Chen, Qi Xuan, and Vladimir Filkov. Sustainability forecasting for apache incubator projects. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1056–1067, 2021.
[27] Didi Surian, Nian Liu, David Lo, Hanghang Tong, Ee-Peng Lim, and Christos Faloutsos. Recommending people in developers' collaboration network. In *2011 18th Working Conference on Reverse Engineering*, pages 379–388. IEEE, 2011.
[28] Stephen H Kan. *Metrics and models in software quality engineering*. Addison-Wesley Professional, 2003.
[29] Claude E. Walston and Charles P. Felix. A method of programming measurement and estimation. *IBM Systems Journal*, 16(1):54–73, 1977.
[30] Jason Tsay, Laura Dabbish, and James Herbsleb. Influence of social and technical factors for evaluating contribution in github. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE 2014, page 356–366, New York, NY, USA, 2014. Association for Computing Machinery.
[31] Raphael Oliveira, Camila Ajala, Davi Viana, Bruno Cafeo, and Awdren Fontão. Developer relations (devrel) roles: An exploratory study on practitioners' opinions. In *Proceedings of the XXXV Brazilian Symposium on Software Engineering*, pages 363–367, 2021.
[32] Benedikt Morschheuser, Lobna Hassan, Karl Werder, and Juho Hamari. How to design gamification? a method for engineering gamified software. *Information and Software Technology*, 95:219–237, 2018.
[33] Amy N Langville and Carl D Meyer. *Google's PageRank and beyond: The science of search engine rankings*. Princeton university press, 2006.
[34] Massimiliano Sassoli de Bianchi. The observer effect. *Foundations of science*, 18(2):213–243, 2013.
[35] Isabel Drost-Fromm and Rob Tompkins. Open source community governance the apache way. *Computer*, 54(4):70–75, 2021.
[36] Paolo Boldi, Massimo Santini, and Sebastiano Vigna. Pagerank as a function of the damping factor. In *Proceedings of the 14th international conference on World Wide Web*, pages 557–566, 2005.
[37] Amos Darko, Albert Ping Chuen Chan, Ernest Effah Ameyaw, Emmanuel Kingsford Owusu, Erika Pärn, and David John Edwards. Review of application of analytic hierarchy process (ahp) in construction. *International journal of construction management*, 19(5):436–452, 2019.
[38] Tomoko Nemoto and David Beglar. Likert-scale questionnaires. In *JALT 2013 conference proceedings*, pages 1–8, 2014.
[39] Patrick Ayoup, Diego Elias Costa, and Emad Shihab. Achievement unlocked: a case study on gamifying devops practices in industry. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1343–1354, 2022.
[40] Yu Huang, Denae Ford, and Thomas Zimmermann. Leaving my fingerprints: Motivations and challenges of contributing to oss for social good. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1020–1032. IEEE, 2021.
[41] Virginia Braun and Victoria Clarke. *Thematic analysis*. American Psychological Association, 2012.

[42] Yvonna S Lincoln and Egon G Guba. *Naturalistic inquiry*. sage, 1985.
[43] Justin P Johnson. Collaboration, peer review and open source software. *Information Economics and Policy*, 18(4):477–497, 2006.
[44] Sean P Goggins, Matt Germonprez, and Kevin Lumbard. Making open source project health transparent. *Computer*, 54(8):104–111, 2021.
[45] Sean Goggins, Kevin Lumbard, and Matt Germonprez. Open source community health: Analytical metrics and their corresponding narratives. In *2021 IEEE/ACM 4th International Workshop on Software Health in Projects, Ecosystems and Communities (SoHeal)*, pages 25–33. IEEE, 2021.
[46] Donald L Thistlethwaite and Donald T Campbell. Regression-discontinuity analysis: An alternative to the ex post facto experiment. *Journal of Educational psychology*, 51(6):309, 1960.
[47] Asher Trockman, Shurui Zhou, Christian Kästner, and Bogdan Vasilescu. Adding sparkle to social coding: an empirical study of repository badges in the npm ecosystem. In *Proceedings of the 40th international conference on software engineering*, pages 511–522, 2018.
[48] Yangyang Zhao, Alexander Serebrenik, Yuming Zhou, Vladimir Filkov, and Bogdan Vasilescu. The impact of continuous integration on other software development practices: a large-scale empirical study. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 60–71. IEEE, 2017.
[49] Théo Zimmermann and Annalí Casanueva Artís. Impact of switching bug trackers: a case study on a medium-sized open source project. In *2019 ieee international conference on software maintenance and evolution (icsme)*, pages 13–23. IEEE, 2019.